

第2章

信息表达与逻辑思维

武汉大学 计算机学院

第二章 信息表达与逻辑思维



- 主题1、 为什么是0与1
- 主题2、 布尔逻辑与逻辑思维
- 主题3、 0与1呈现的数值世界
- 主题4、 0与1呈现的文字世界
- 主题5、 0与1呈现的声色世界



课前游戏-1



- ✓ 从下列姓氏中选中一个，默默记下
- ✓ 从后续给出的姓氏中，判断是否出现了你选定的姓氏

赵、钱、孙、李

周、吴、郑



课前游戏-1



1/3 是否有你选定的姓氏出现？

赵、孙、周、郑



课前游戏-1



2/3 是否有你选定的姓氏出现？

钱、孙、吴、郑



课前游戏-1



3/3 是否有你选定的姓氏出现？

李、周、吴、郑



2.1 为什么是0与1 ——二进制历史追溯



是谁发明了二进制？

- ✓ 莱布尼兹？（论二进制算术）
- ✓ 古老的中国人？（易经）



周文王姬昌
前 1152—前 1056

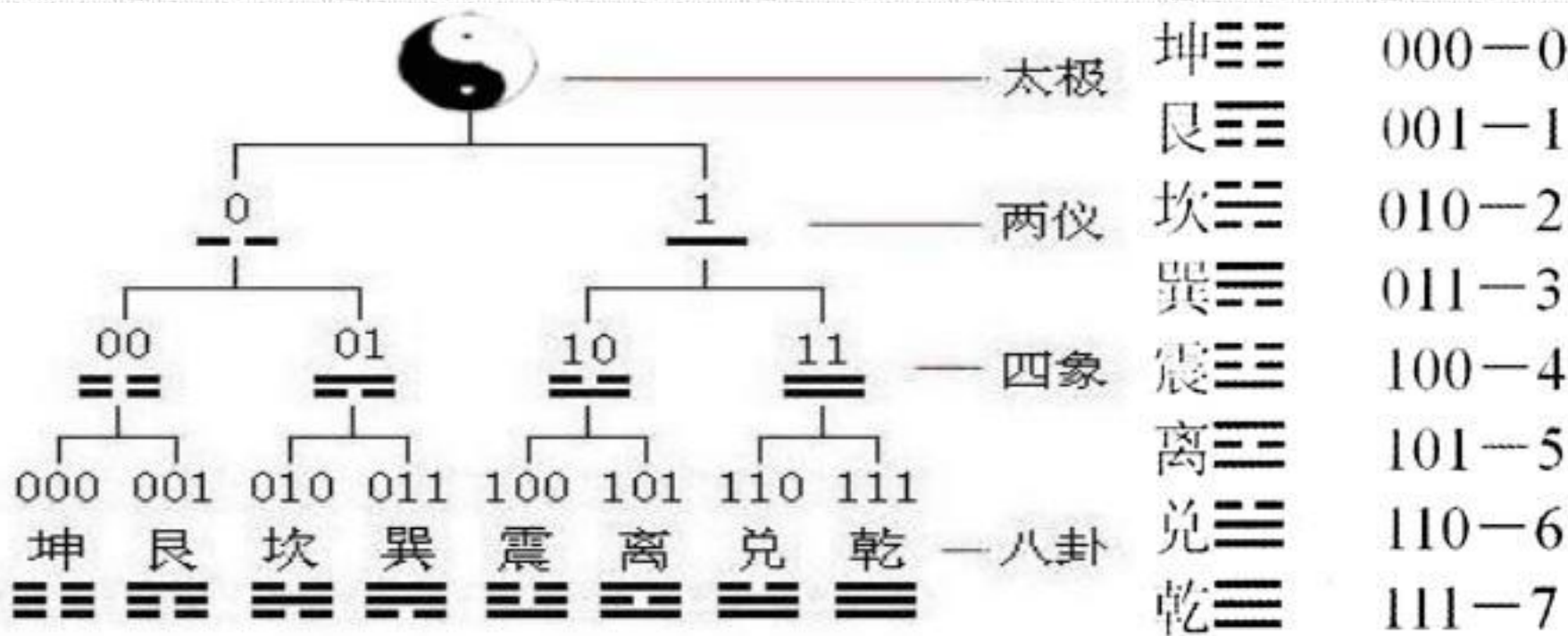
莱布尼兹从《易经》的八卦中想到了二进制， 1703年发表了《论二进制算术》，提出了二进制的概念。



莱布尼兹
1646-1716



2.1 为什么是0与1 ——二进制历史追溯



2.1 为什么是0与1 ——二进制历史追溯



冯·诺伊曼
(1903-1957)

- 冯·诺伊曼将二进制引入了计算机。
- 1和0分别代表电路的开和关、电压的高和低等状态。



二进制



- ◆ 信息复制的精确性
- ◆ 运算规则简单 ($R(R+1)/2$)
- ◆ 电子线路制造计算机成为可能

我认识她已有1111年了。



2.1 为什么是0与1 ——二进制历史追溯



计算机选择了 0与1

- 容易被物理器件所实现
- 可靠性高
- 运算规则简单
- 与逻辑量相吻合



2.1 为什么是0与1 ——二进制游戏原理



	1	
3	5	7
9	11	13
15	17	19
21	23	25
27	29	31

	2	
3	6	7
10	11	14
15	18	19
22	23	26
27	30	31

	4	
5	6	7
12	13	14
15	20	21
22	23	28
29	30	31

	8	
9	10	11
12	13	14
15	24	25
26	27	28
29	30	31

	16	
17	18	19
20	21	22
23	24	25
26	27	28
29	30	31

× × × × 1



	1	
3	5	7
9	11	13
15	17	19
21	23	25
27	29	31



× × × 1 ×



	2	
3	6	7
10	11	14
15	18	19
22	23	26
27	30	31



× × 1 × ×



	4	
5	6	7
12	13	14
15	20	21
22	23	28
29	30	31



× 1 × × ×



8
9 10 11
12 13 14
15 24 25
26 27 28
29 30 31



1 × × × ×



	16	
17	18	19
20	21	22
23	24	25
26	27	28
29	30	31



抽出来的纸片里写在最上方的数都加起来，它便是你的观众所猜的那个数！比如，观众心里想的数是21，那他抽出的纸片必定是（Ⅰ）、（Ⅲ）、（Ⅴ），这几张纸片的上端的数字分别为1、4、16，因而你观众所想的数是：

$$1 + 4 + 16 = 21$$

这似乎是神奇的，其实道理也很简单，认真观察一下就知道，纸片（Ⅰ）上所有的数用二进制写都是形如

$$\times \times \times \times 1 \quad (2) \qquad 1$$

的数，而纸片（Ⅱ）、（Ⅲ）、（Ⅳ）、（Ⅴ）上的数则分别形如：

$$\times \times \times 1 \times \quad (2) \qquad 2$$

$$\times \times 1 \times \times \quad (2) \qquad 4$$

$$\times 1 \times \times \times \quad (2) \qquad 8$$

$$1 \times \times \times \times \quad (2) \qquad 16$$

如果某一数字在（Ⅰ）、（Ⅲ）、（Ⅴ）中出现，而不在（Ⅱ）、（Ⅳ）中出现，那么此数必为 $10101_{(2)} = 16 + 0 + 4 + 0 + 1 = 21$ 。上面式子中的16,4,1,我们已经用隐蔽的方式，写在相应纸片的上端，不必游戏者去临时换算。

“猜数”游戏可以改头换面，变成一种相当精彩的小魔术：“猜百姓”。在魔术中不见任何一个数字，更无需做什

2.2 布尔逻辑与逻辑思维 — 0与1的运算



- 算术运算

(1) 加法:	$0+0=0$	$1+0=1$	$0+1=1$	$1+1=10$
(2) 减法:	$0-0=0$	$1-0=1$	$1-1=0$	$10-1=1$
(3) 乘法:	$0\times 0=0$	$0\times 1=0$	$1\times 0=0$	$1\times 1=1$
(4) 除法:	$0\div 1=0$	$1\div 1=1$		



2.2 布尔逻辑与逻辑思维 — 0与1的运算



- 逻辑运算

① 逻辑加法

$$0 \vee 0 = 0$$

$$0 \vee 1 = 1$$

$$1 \vee 0 = 1$$

$$1 \vee 1 = 1$$

② 逻辑乘法

$$0 \wedge 0 = 0$$

$$0 \wedge 1 = 0$$

$$1 \wedge 0 = 0$$

$$1 \wedge 1 = 1$$

③ 逻辑否定

非 0 等于 1

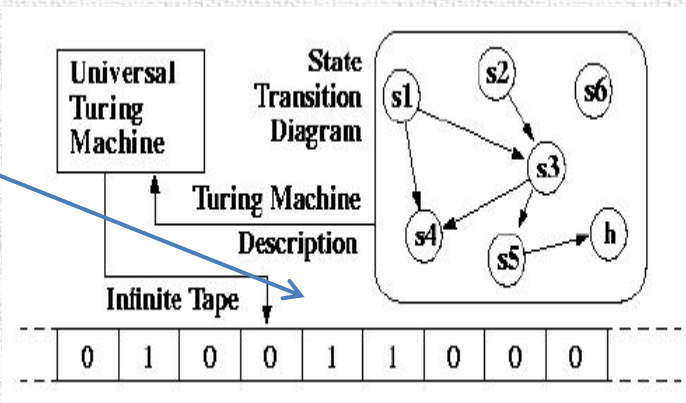
非 1 等于 0



2.2 布尔逻辑与逻辑思维 — 0与1的逻辑

Head

Tape



State table

状态ID	读取值	写入值	读写头	下个状态
S1	00	00	左移	S1
S1	01	01	左移	S1
S1	10	01	左移	S1
S1	11	10	左移	S1
S1	B	—	H	S2

二进制数: $1 + 0 = ?$

$1 + 1 = ?$

X	Y	Z
0	0	0
0	1	1
1	0	1
1	1	0

$$Z = (X \wedge \neg Y) \vee (\neg X \wedge Y)$$

2.2 布尔逻辑与逻辑思维 — 布尔逻辑运算



- 布尔常量：真 T (true)，假 F (false)

i.e. 今天下雨。 $a^2 \geq 0$ 。

- 合取，与 $\wedge$ (conjunction, and)

- $x \wedge y = 1$ (T) iff $x = y = 1$ (T)

- 析取，或 $\vee$ (disjunction, or)

- $x \vee y = 0$ (F) iff $x = y = 0$ (F)

- 非 $\neg$ (negation, not)

- $\neg x = 1$ (T) iff $x = 0$ (F)

- 蕴含 $\rightarrow$ (implication)

- $(x \rightarrow y) = 1$ (T) iff $x = 0$ (F) or $y = 1$ (T)

i.e. 山无陵，江水为竭，冬雷震震，夏雨雪，
天地合，乃敢与君绝。

- 异或 $\oplus$ (exclusive or)

- $x \oplus y = 1$ (T) iff $x \neq y$



2.2 布尔逻辑与逻辑思维 — 布尔逻辑运算

916

真值表

表征逻辑事件输入
和输出之间全部可
能状态的表格

x	y	$x \wedge y$	$x \vee y$	$x \rightarrow y$	$x \oplus y$
0	0	0	0	1	0
0	1	0	1	1	1
1	0	0	1	0	1
1	1	1	1	1	0

2.2 布尔逻辑与逻辑思维 一计数制



例如: $(10110.1)_2$

$$= 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1}$$
$$= (22.5)_{10}$$

任意一个二进制数B, 可以展开成多项式之和, 即

$$B = b_{n-1}2^{n-1} + b_{n-2}2^{n-2} + \dots + b_12^1 + b_02^0 +$$
$$b_{-1}2^{-1} + \dots + b_{-m}2^{-m}$$


2.2 布尔逻辑与逻辑思维 — 计数制



- 数码
 - 每种数制固定的符号集。例如，十进制有0、1、…、9 共10个数码，二进制采用0、1 共2 个数码。
- 基
 - 计数制所使用的数码个数，称为“基”。十进制的基为10，逢十进一；二进制的基为2，逢二进一。
- 权
 - 处于不同位置的数符所代表的值不同，与它所在的位置的权值有关。
 - 例如，十进制数5555.555 可表示为：
$$5555.555 = 5 \times 10^3 + 5 \times 10^2 + 5 \times 10^1 + 5 \times 10^0 + 5 \times 10^{-1} + 5 \times 10^{-2} + 5 \times 10^{-3}$$



2.2 布尔逻辑与逻辑思维 — 计数制



十进制：

- 十进制的10 个数码本身代表了确定的值，当它们处在十进制数中的不同位置时，实际值是不同的，等于本身值乘以一个以10 为底数的指数值（权）。
- **【例】** $576 = 5 \times 10^2 + 7 \times 10^1 + 6 \times 10^0$ 。



2.2 布尔逻辑与逻辑思维 — 计数制



二进制：

- 二进制的数码为0和1，基数为2，并按“逢二进一”的原则进行计数。
- **【例】** $(1011101)_2 = 2^6 + 2^4 + 2^3 + 2^2 + 2^0 = 64 + 16 + 8 + 4 + 1 = (93)_{10}$ 。

为了区分不同进制的数，在书写时有两种常用的方法。

一种是将数字用括号加下标数字的方式来表明该数采用哪种进制，如 $(1011101)_2$ 、 $(93)_{10}$ 。

另一种是在数的后面加上不同的字母表示进制，D表示十进制，B表示二进制，O表示八进制，H表示十六进制，如1100B、AC72H。



2.2 布尔逻辑与逻辑思维 — 计数制



- 八进制和十六进制：

- 八进制有8个数码，即0、1、…、7，基数为8，按“逢八进一”的原则进行计数。
- 十六进制有16个数码，即0、1、…、9、A、B、C、D、E、F，基数为16，按“逢十六进一”的原则进行计数。其中，A表示10，B表示11，……，F表示15。



2.2 布尔逻辑与逻辑思维 一计数制



表 2.1 计算机中常用的几种进位制数的表示

进位制	二进制	八进制	十进制	十六进制
规则	逢二进一	逢八进一	逢十进一	逢十六进一
基数	$r=2$	$r=8$	$r=10$	$r=16$
数符	0, 1	0, 1, 2, ..., 7	0, 1, ..., 9	0, 1, ..., 9, A, B, C, D, E, F
权	2^i	8^i	10^i	16^i
形式表示	B	O	D	H



2.2 布尔逻辑与逻辑思维 一进制转换



1. 各种进制转十进制：按权展开法

【例】二进制转十进制。

$$(1011101)_2 = 2^6 + 2^4 + 2^3 + 2^2 + 2^0 = 64 + 16 + 8 + 4 + 1 = (93)_{10}$$

【例】八进制转十进制。

$$(2613)_8 = 2 \times 8^3 + 6 \times 8^2 + 1 \times 8^1 + 3 \times 8^0 = (1419)_{10}$$

【例】十六进制转十进制。

$$(3AF)_{16} = 3 \times 16^2 + 10 \times 16^1 + 15 \times 16^0 = (943)_{10}$$



2.2 布尔逻辑与逻辑思维 — 进制转换



2. 十进制转二进制

(1) 整数部分：“除2取余”，直到商为0。

$$(26)_{10} = (11010)_2$$

除 2	26	
除 2	13	余 0
除 2	6	余 1
除 2	3	余 0
除 2	1	余 1
	0	余 1



2.2 布尔逻辑与逻辑思维 — 进制转换



2. 十进制转二进制

(2) 纯小数部分：“乘2取整”，直到小数部分为0 或达到计算精度要求。

$$\begin{array}{r} 0.3125 \\ \times 2 \\ \hline 0.625 \\ \times 2 \\ \hline 1.25 \\ \times 2 \\ \hline 0.5 \\ \times 2 \\ \hline 0 \end{array}$$

整数部分为 0

整数部分为 1，取纯小数部分 0.25 再计算

整数部分为 0

整数部分为 1，小数部分为 0，终止

$$(0.3125)_{10} = (0.0101)_2$$



2.3 0与1的数值世界——整数表达



- 整数可以分为无符号整数和有符号整数两类。
 - 无符号整数的所有二进制位全部用来表示数值的大小。
 - 有符号整数用最高位表示数的正负号，而其他位表示数值的大小。
- 计算机中对有符号数的不同运算采用不同的编码方法，主要有原码、反码和补码3种。



2.3 0与1的数值世界——整数表达



- 原码

- 最高位表示符号位，正数为0、负数为1。
- 例如，19的二进制数为10011，19与-19的原码如下(计算机存储整数一般用长度为16位或32位的二进制位，若用16位，最高位为符号位，后15位为表示真值):

[19]原 = 0000 0000 0001 0011

[-19]原 = 1000 0000 0001 0011



2.3 0与1的数值世界——整数表达



- 反码

- 正数的反码等于其原码，负数的反码是将其原码除符号位外，其余部分全部按位取反。

$[19]_{\text{反}} = [19]_{\text{原}} = 0000\ 0000\ 0001\ 0011$

$[-19]_{\text{原}} = 1000\ 0000\ 0001\ 0011$

$[-19]_{\text{反}} = 1111\ 1111\ 1110\ 1100$



2.3 0与1的数值世界——整数表达



- 补码

- 正数的补码等于其原码，负数的补码等于其反码末位加1。

$[19]_{\text{补}} = [19]_{\text{反}} = [19]_{\text{原}} = 0000\ 0000\ 0001\ 0011$

$[-19]_{\text{原}} = 1000\ 0000\ 0001\ 0011$

$[-19]_{\text{反}} = 1111\ 1111\ 1110\ 1100$

$[-19]_{\text{补}} = 1111\ 1111\ 1110\ 1101$



2.3 0与1的数值世界——整数表达



- 补码

— 计算机中将减法转化为加法运算的基础——减去一个数，等于加上这个数的补码。





补码的用途 —— 变减法为加法



$$10-4=6 \text{ 倒拔}$$

$$10+8=6 \text{ 顺拔}$$

$$10+8=12+6$$

模

8与-4对模12互为补数

2.3 0与1的数值世界——整数表达



- 补码

- 假设用16位二进制运算，最高位为符号位，两个数分别为十进制的 $a=11$ 和 $b=-10$ ，则 a 转换为二进制的补码为00001011； b 的二进制原码10001010，反码为11110101，补码为11110110。则使用补码计算 a 和 b 之和：

0000 0000 0000 1011	+11的补码，符号位为0
+ 1111 1111 1111 0110	-10的补码，符号位为1
1 0000 0000 0000 0001	产生的进位丢掉



2.3 0与1的数值世界 --0的表达



$$(+0) = (00000000)_{\text{原}}$$

$$(-0) = (10000000)_{\text{原}}$$

$$(+0) = (00000000)_{\text{反}}$$

$$(-0) = (11111111)_{\text{反}}$$

$$(+0) = (00000000)_{\text{补}}$$

$$(-0) = (11111111)_{\text{反}} + 1 = (\textcolor{red}{1}00000000)_{\text{补}}$$

原码的零有两种表示：

正零为0.0...0；负零为1.0...0

反码的零有两种表示：

正零为0.0...0，负零为1.1...1。

补码的零有唯一的表示，即：

0.0...0。



2.3 0与1的数值世界——补码目的



- 所以补码的设计目的是:

(1)使符号位能与有效值部分一起参加运算,从而简化运算规则.

(2)使减法运算转换为加法运算,进一步简化计算机中运算器的线路设计

所有这些转换都是在计算机的最底层进行的,而在我们使用的汇编、C等其他高级语言中使用的都是原码。



2.3 0与1的数值世界——减法变加法



变
减
法
为
加
法

例如，在**五位**加法器中实现 **12-7** 的运算：

$$\because [-7]_{\text{原}} = 1\ 0\ 1\ 1\ 1 \quad [-7]_{\text{补}} = 1\ 1\ 0\ 0\ 1$$

$$[12]_{\text{补}} =$$

$$[-7]_{\text{补}} =$$

丢失

1

0	1	1	0	0
1	1	0	0	1
0	0	1	0	1

+

$$\therefore 12-7=12+25=5 \quad (\text{丢失模 } 32)$$



2.3 0与1的数值世界——减法变加法



- ❖ 原则： $X_{\text{补}} + Y_{\text{补}} = (X + Y)_{\text{补}}$
- ❖ 用于做减法时方便，将减法变成做加法，而且符号也参与运算。
- ❖ 例如： $9_{\text{D}} - 6_{\text{D}} = (?)_{\text{D}}$

转变为二进制数，即是求：

$$1001\text{B} - 0110\text{B} = (?)\text{B}$$

$$1001\text{B} - 0110\text{B} = 1001\text{B} + (-0110)\text{B}$$

$$\text{反码: } 1001\text{B} \quad 11001\text{B}$$

$$X_{\text{补}} \quad Y_{\text{补}} = (X + Y)_{\text{补}}$$

$$\text{补码: } 1001\text{B} \quad 11010\text{B} = 10011\text{B}$$

$$\text{原码: } 1001\text{B} \quad 10110\text{B} \quad 0011\text{B}$$

$$\text{所以: } 1001\text{B} - 0110\text{B} = (0011)\text{B} = 3_{\text{D}}$$

符号位只保留1位，
溢出的位自动丢弃



2.3 0与1的数值世界——减法变加法



❖ 例2: $0.1101B - 0.1011B = (?)B$

$$0.1101B - 0.1011B = 0.1101B + (-0.1011)B$$

反码: $0.1101B$ $1.0100B$

$$X_{\text{补}} \quad Y_{\text{补}} = (X+Y)_{\text{补}}$$

补码: $0.1101B$ $1.0101B =$ $1\ 0.0010B$

原码: $0.1001B$ $1.1011B$ $0.0010B$

所以: $0.1101B - 0.1011B = (0.0010)B$

符号位只保留1位，
溢出的位自动丢弃

2.3 0与1的数值世界——实数表达



计算机是有限物理存储空间的机器，因此表示数就需要考虑数的长度。一般计算机中的实数有两种常用表示格式：定点数和浮点数。



2.3.1 实数表达 一定点数



1、定点数

定点格式表示的数值范围有限，在计算机输出中一般用固定长度（如16位或32位二进制）表示，同时将小数点固定在某一个位置。为了处理方便，一般分为定点纯小数和定点纯整数，前文所讲的原码、反码、补码表示的就是定点纯整数。



2.3.1 实数表达 一定点数



(1) 定点纯小数

定点纯小数是指小数点准确固定在数据某一个位置上的小数。一般把小数点固定在最高数据位的左边，小数点前边再设一位符号位。按此规则，任何一个小数都可以写成：

$$N = N_s N_{-1} N_{-2} N_{-3} \dots N_{-m}, \quad N_s \text{——符号位}$$

即在计算机中用 $m+1$ 个二进制位表示一个小数，最高（最左）一个二进制位表示符号（如用0表示正号，1表示负号），后面的 m 个二进制位表示该小数的数值。对用 $m+1$ 个二进制位表示的小数来说，其值的范围： $|N| \leq 1 - 2^{-m}$ 。



2.3.1 实数表达——定点数



(2) 定点纯整数

定点纯整数可以认为它是小数点定在数值最低位右面的一种表示法。整数分为带符号和不带符号两类。对带符号的整数，符号位放在最高位。可以写成：

$$N = N_s N_n N_{n-1} N_{n-2} \dots N_2 N_1 \quad N_s \text{——符号位}$$

对于用 $n+1$ 位二进制位表示的带符号整数，其值的范围为： $|N| \leq 2^{n-1}$ 。

对于不带符号的整数，所有的 $n+1$ 个二进制位均看成数值，此时数值表示范围为： $0 \leq N \leq 2^{n+1}-1$ 。



2.3.1 实数表达 一定点数



在计算机中,一般用8位、16位和32位等表示数据。一般定点数表示的范围和精度都较小,在数值计算时,大多数采用浮点数。

定长数据格式要求的处理硬件比较简单,但它的表示范围受到限制,与所需表示的数值取值范围相差悬殊,给存储和计算带来诸多不便,因此出现了浮点表示法。



2.3.2 实数表达 — 浮点数



2、浮点数

浮点表示法，即小数点的位置提浮动的，其思想来源于科学计数法。

任何实数都可以表示为 $\pm 10^{\pm m} \times N$ 。其中 m 为整数， N 为大于1小于10的正数。如：

$$-583310029120000 = -10^{+14} \times 5.8331002912$$

$$0.0000000345 = +10^{-8} \times 3.45$$



2.3.2 实数表达 —— 浮点数



2、浮点数

二进制数也有科学计数法，任何一个二进制数都可以表示为 $\pm 2^{\pm m} \times N$ 。

如下列二进制数可以表示成（指数也是二进制数）：

$$-11011.1101 = -2^{+101} \times 0.110111101$$

$$0.00101 = +2^{-10} \times 0.101$$



2.3.2 实数表达 —— 浮点数



2、浮点数

计算机采用将符号、指数部分（阶码）和尾数部分分段表示的方法来表示实数，其目的是以有限的二进制位尽可能保持有效位数。浮点数能表示的数值范围很大，要求的处理硬件比较复杂。浮点数分数符、阶码和尾数三个部分。



2.3.2 实数表达 — 浮点数

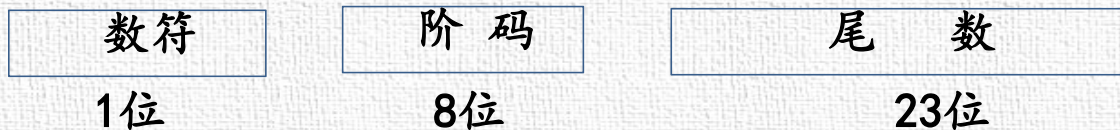


- 阶码

- 阶码即指数部分，用于表示小数点在该数中的位置，它是一个定点纯整数。

- 尾数

- 尾数用于表示数的有效数值，用定点纯小数表示。一般选择32位（单精度）或64位（双精度）二进制表示一个浮点数。32位浮点数格式如下：



2.3.2 实数表达 — 浮点数



假设浮点数采用8位二进制进行存储，阶码占3位，尾数占5位，阶码及尾数都带符号位

阶码符号	阶码	阶码	尾数符号	尾数	尾数	尾数	尾数
?	?	?	?	?	?	?	?
?	?	?	?	?	?	?	?

该浮点表示法所能表示的最大值及最小值是多少？



2.3.2 实数表达 — 浮点数



假设浮点数采用8位二进制进行存储，阶码占3位，尾数占5位，阶码及尾数都带符号位，要求尾数的绝对值必须 $\geq 1/2$

阶码符号	阶码	阶码	尾数符号	尾数	尾数	尾数	尾数
+	1	1	+	1	1	1	1
+	1	1	-	1	1	1	1

$$0.1111 \times 2^3 = 111.1 = 7.5 \quad (\text{最大值 } 7.5)$$

$$-0.1111 \times 2^3 = -111.1 = -7.5 \quad (\text{最小值 } -7.5)$$

该浮点表示法所能表示的最大值及最小值是多少？



2.4 0与1的文字世界 --ASCII



- 为了信息交换中的统一性，人们已经建立了一些字符编码标准，目前国际上广泛使用的是ASCII码。
- 标准ASCII码的编码，用一个字节中的低7位对应一个字符的编码，可以为128种不同的符号编码。
- 计算机以一个字节存储ASCII码表中各字符的编码信息，最高位补0。



2.4 0与1的文字世界--ASCII



- 部分ASCII码表

ASCII值	控制字符	ASCII值	控制字符	ASCII值	控制字符	ASCII值	控制字符
0	NUT	32	(space)	64	@	96	`
1	SOH	33	!	65	A	97	a
2	STX	34	”	66	B	98	b
3	ETX	35	#	67	C	99	c
4	EOT	36	\$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	,	71	G	103	g
8	BS	40	(	72	H	104	h
9	HT	41	)	73	I	105	i
10	LF	42	*	74	J	106	j



2.4 0与1的文字世界——ASCII



- 计算机是如何显示这些字符的呢？
- 以字符“A”为例，写在一个8行8列的网格上，就得到了该字符的点阵信息（笔画经过的方格为1，其他方格为0）。一行8个点用一个字节表示，8个字节就可以存储一个字符的点阵信息，称为该字符的字模。
- 所有这些可显示字符的字模，顺序存放在计算机中，就组成了ASCII字的8*8字库。同理，如果通过8*16的网格获取各字符的点阵信息，可组成ASCII字符的8*16字库。



2.4 0与1的文字世界--ASCII



- 字符的编码信息，与该字符的字模在字库中所存放的地址存在对应关系。当我们在输入一个文档时，按了“A”这个按键，计算机就存入一个字节的的信息，其内容为01000001。
- 该代码间接表示字符“A”的点阵信息，因为在显示时，系统要根据这个代码找到“字符A”的字模，将字模中的“1”用前景色显示，将字模中的“0”用背景色显示，屏幕上就会显示出“A”的图样。



2.4 0与1的文字世界——汉字编码



- 在我国应用计算机，当然需要计算机处理汉字，中文数据也采用二进制编码来表示。汉字由于数量大，用8位二进制编码方式无法表示全部汉字，一般采用多个字节表示。
- 据统计，常用的汉字有四五千个。汉字字符集是一个很大的集合，要编码常用的汉字至少需要两个字节作为编码的形式。事实上，两个字节可以表示 $2^{16}=65536$ 种不同的符号，但考虑到汉字编码与其他国际通用编码（如ASCII码）的关系，我国国家标准局采用了加以修正的两字节汉字编码方案，即只用了两个字节的低7位。这个方案可以容纳 $2^{14}=16384$ 种不同的符号。



2.4 0与1的文字世界——汉字编码



1、国标码、区位码、机内码

- 1981年，为了使每一个汉字有一个全国统一的代码，我国国家标准局颁布了第一个汉字编码的国家标准：《信息交换用汉字编码字符集——基本集》，简称国标码，代号为GB2312-80。这个字符集是我国中文信息处理技术的发展基础，也是目前国内所有汉字系统的统一标准。



2.4 0与1的文字世界——汉字编码



(1) 区位码

为了便于使用，GB2312-80的国家标准将其中的汉字和其他符号按照一定的规则排列成为一个大的表格，在这个表格中，每一（横）行称为一个“区”，每一（竖）列称为一个“位”，整个表格共有94区，每区有94位，并将“区”和“位”用十进制数字进行编号：即区号为01-94，位号为01-94。



2.4 0与1的文字世界——汉字编码



(1) 区位码

第01-09区：分别存放了682个标点符号、运算符号、制表符号、数字、序号、英文字母、俄文字母、日文假名、希腊字母、汉语拼音字母、汉语注音字母等；

第10-15区：有待扩展的空白区；

第16-55区：按照汉语拼音的顺序依次存放了3755个一级汉字（最常用的汉字）；

第56-87区：按照部首顺序依次存放了3008个二级汉字（次常用的汉字）；

第88区以后：有待扩展的空白区。

一、二级汉字合计共6763个。



2.4 0与1的文字世界——汉字编码



(2) 国标码

如果知道某个汉字的区位码，只要将区号和位号分别加上32（若十六进制表示则加上20H），就可以得到该汉字的国标码。如“啊”字的区位码是1601（十六进制为1001H），国标码为4833（十六进制数为3021H）。

国标码高位 = 区码 + 20H

国标码低位 = 位码 + 20H



2.4 0与1的文字世界——汉字编码



(3) 机内码

“限”的区位码：4762, 对应的16进制：2F3EH

“限”国标码：7994, 对应16进制：4F5EH

如果是国标码存储，则两个字节的二进制信息如下：

0	1	0	0	1	1	1	1		0	1	0	1	1	1	1	0
---	---	---	---	---	---	---	---	--	---	---	---	---	---	---	---	---

但对照前文的ASCII码表，这两个字节的信息分别对应的是“O”和“^”。

系统该如何分辨呢？



2.4 0与1的文字世界——汉字编码



(3) 机内码

所以，计算机不能将国标码作为汉字在计算机中的机内码，于是，国家标准规定将汉字国标码的每个字节的最高位统一规定为“1”，作为识别汉字代码的标志，这就形成了机内码。

“限”机内码二进制信息如下：

1	1	0	0	1	1	1	1		1	1	0	1	1	1	1	0
---	---	---	---	---	---	---	---	--	---	---	---	---	---	---	---	---

机内码高位 = 国标码高位 + 80H,
机内码低位 = 国标码低位 + 80H。



2.4 0与1的文字世界——汉字编码



2、输入码、字型码

(1) 输入码

由于汉字有数以万计，电脑键盘不可能为每一个汉字而造一个按键。因此，人们需要替汉字编输入码，用数个键来输入一个汉字。不同的输入法得到的输入码的方法不同，它与汉字输入法的处理过程有关。汉字的单字输入分为几类：音码、形码、形音码、音形码、无理码等。所有的输入码的“终极目标”就是把人们的输入转化成汉字的机内码。



2.4 0与1的文字世界——汉字编码



2、输入码、字型码

(2) 字型码

为了能在屏幕或打印机等设备上输出汉字，需要汉字的字型。任何一个编码字条都需要有相应字形图样的“库”。字库通常有两种表示法，点阵表示法和矢量表示法。

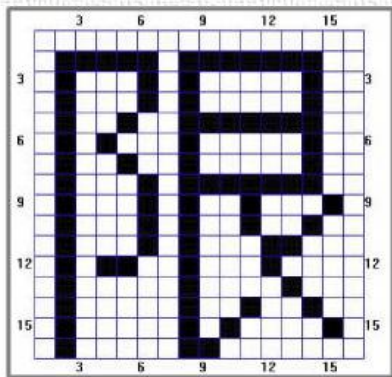


2.4 0与1的文字世界——汉字编码

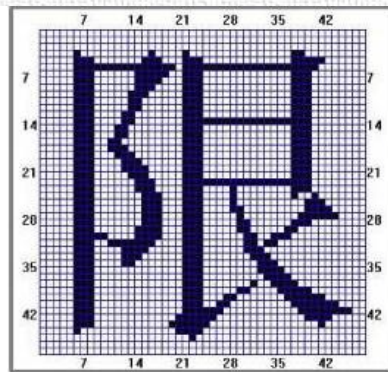


①点阵字库

点阵是一种数字化字库的格式，每个字形以黑白像素点矩阵组成。常用点阵字库规格有12点阵、14、16、24、32、48点阵。



(图一)



(图二)



2.4 0与1的文字世界——汉字编码



①点阵字库

对一个16点阵字形而言，一行就需要用2个字节（16位）来描述，它的存贮信息量是 $2 \times 16 = 32$ 字节。而一个48点阵字形的存贮信息量是 $6 \times 48 = 288$ 字节。

通常在电脑里每个点阵字库有几千上万个字组成，一个字库的存贮信息量可以跟据字库规格和字数计算出来，随着点阵矩阵的增加而增加。

一般16以下的点阵用于各种电子设备的屏幕显示：如计算机、手机、好易通、记录仪、MP3等。24以上点阵用于针式打印机或激光打印机输出。



2.4 0与1的文字世界——汉字编码



② 矢量字库

通过矢量方法来描述字符的轮廓，这种字体也称为轮廓字体。曲线是由数学曲线表达的，并由一组程序指令实现字符外形（轮廓）输出。

在Windows操作系统中使用的True Type技术就是汉字的矢量表示方式。

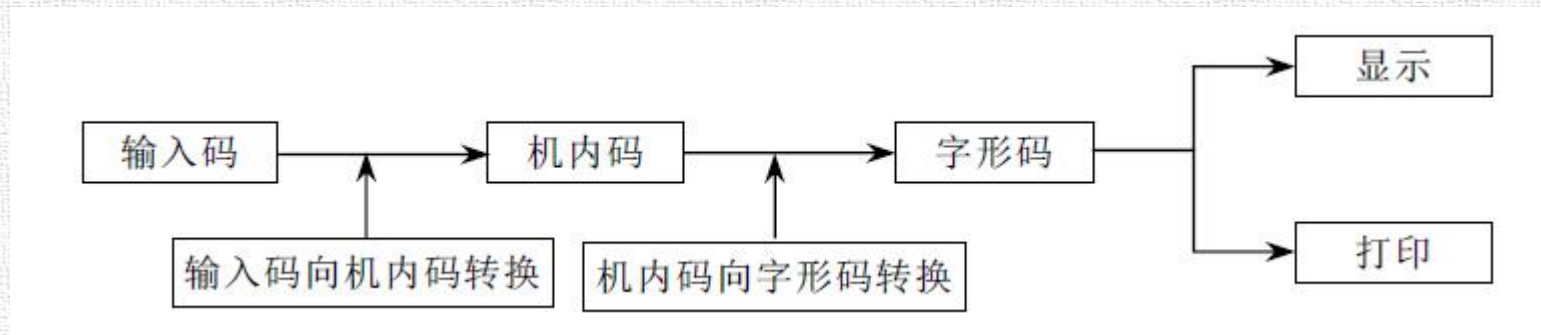
TrueType Font，是苹果公司及微软公司共同制定的一种字体格式，它采用二次曲线来描述字形轮廓，具有字库信息小，字形可以随意缩放、变形而不失真等优点。



2.4 0与1的文字世界——汉字编码



3、汉字信息的输入、储存与显示



2.4 0与1的文字世界--Unicode编码



世界上存在着多种编码方式，同一个二进制数字可以被解释成不同的符号。因此，要想打开一个文本文件，就必须知道它的编码方式，否则用错误的编码方式解读，就会出现乱码。为什么电子邮件常常出现乱码？就是因为发信人和收信人使用的编码方式不一样。

可以想象，如果有一种编码，将世界上所有的符号都纳入其中。每一个符号都给予一个独一无二的编码，那么乱码问题就会消失。这就是Unicode，就像它的名字所表示的，这是一种所有符号的编码。



2.4 0与1的文字世界 --Unicode编码



Unicode是一个很大的集合，现在的规模可以容纳100多万个符号。每个符号的编码都不一样，比如，U+0639表示阿拉伯字母Ain，U+0041表示英语的大写字母A，U+4E00表示汉字“一”。具体的符号对应表，可以查询 unicode.org，或者专门的汉字对应表。Unicode 16编码里面已经包含了GB18030里面的所有汉字（27484个字），Unicode标准准备把康熙字典的所有汉字放入到Unicode 32bit编码中。



2.4 0与1的文字世界 --Unicode编码



Unicode只是一个符号集，它只规定了符号的二进制代码，却没有规定这个二进制代码应该如何存储。比如，汉字“一”的Unicode是十六进制数4E00，转换成二进制数足足有15位（100111000000000），也就是说这个符号的表示至少需要2个字节。而表示其他更大的符号，可能需要3个字节或者4个字节，甚至更多。



2.4 0与1的文字世界 --Unicode编码



这里就有两个问题：

一，如何才能区别 Unicode和 ASCII?

二，如果Unicode统一规定，每个符号用三个或四个字节表示，那么每个英文字母前都必然有二到三个字节是0，这对于存储空间来说是极大的浪费，文本文件的大小会因此大出二三倍。

它们造成的直接结果是：出现了Unicode的多种存储方式，也就是说有许多种不同的二进制格式，可以用来表示 Unicode。另外 Unicode在很长一段时间内无法推广，直到互联网的出现。网络上流行的Utf-8就是Unicode编码的一类应用。



2.5 0与1的声色世界——音频

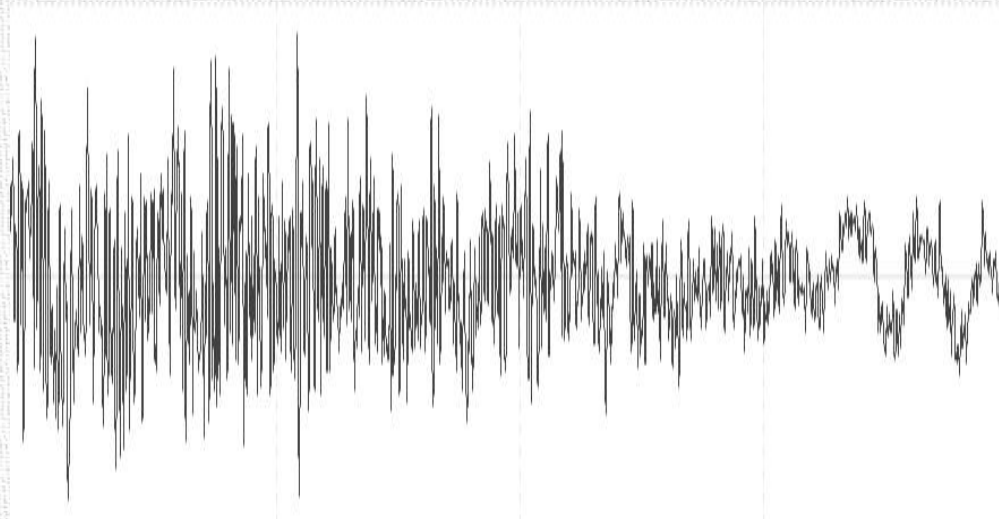


1、音频信号的特征

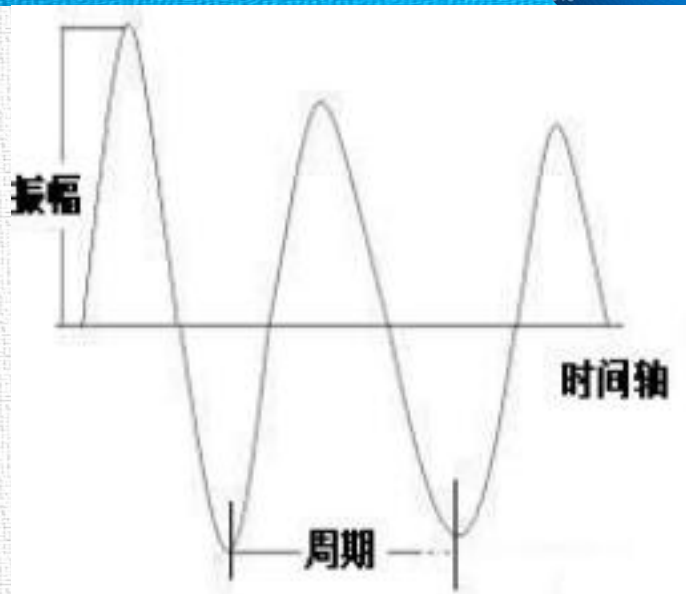
- 音频信号所携带的信息大体上可分为语音、音乐和音响3类。语音是指具有语言内涵和人类约定的特殊媒体；音乐是规范的符号化了的的声音；而音响指其他自然声音，如动物的叫声、机器的轰鸣声、风雨雷电声等。



2.5 0与1的声色世界——音频数字化



一段音乐



把时间轴放大后的一小段



2.5 0与1的声色世界——音频数字化



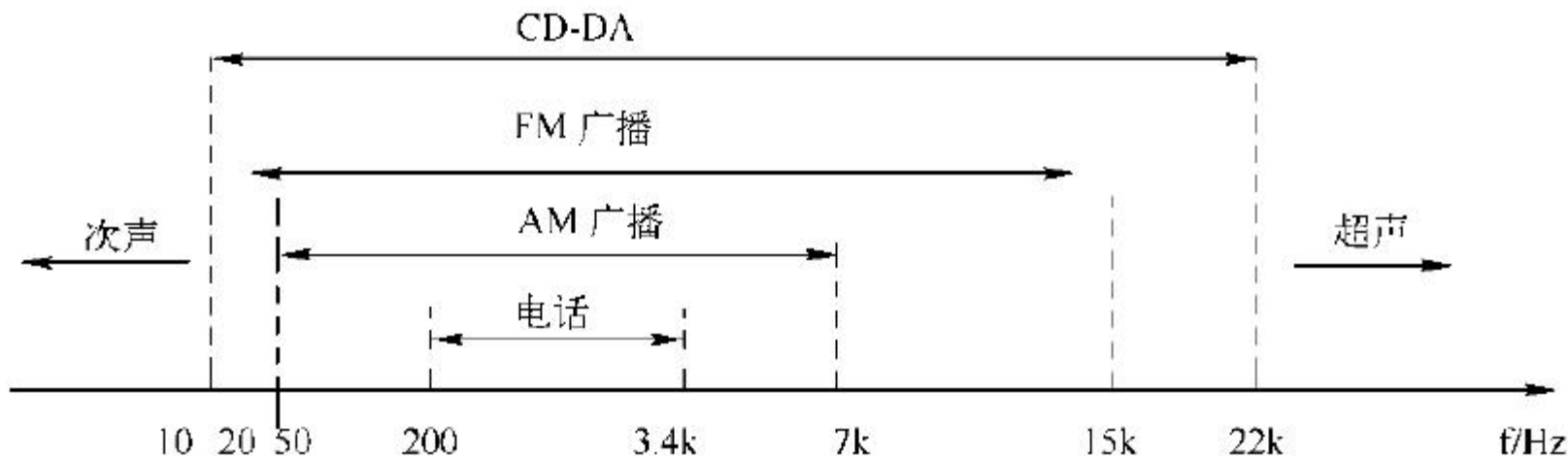
- 声波可以用一条连续的曲线来表示，它在时间和幅度上都是连续的，称为模拟音频信号。磁带和老式密纹唱片上记录的是模拟声音信号。AM、FM广播信号也是模拟信号。
- 这条曲线可以分解成一系列正弦函数的线性叠加。声音强弱体现在声波的振幅大小上。音调的高低体现在声音频率上。音色由声音频率组成成份决定。



2.5 0与1的声色世界——音频数字化



- 声音频率可分为三种：次声（频率低于20Hz），超声（频率高于20kHz），可听声（频率在20Hz—20kHz之间）。



2.5 0与1的声色世界——音频数字化



2、声音数字化过程

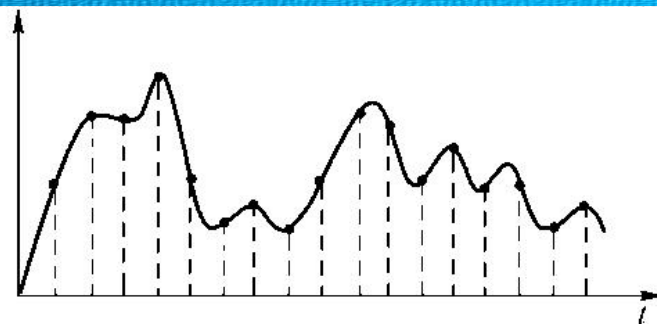
- 采样：每隔一定的时间测量一次声音信号的幅值，把时间连续的模拟信号转换成每隔一定时间间隔的信号样本值序列。
- 量化：用有限个幅度近似表示原来在时间上连续变化的幅度值，把模拟信号的连续幅度变为有限数量、有一定时间间隔的离散值。
- 编码：经过采样和量化以后的数字信号以一定的方式进行编码，形成计算机内部运行的数据。经过编码后的声音信号就是数字音频信号。



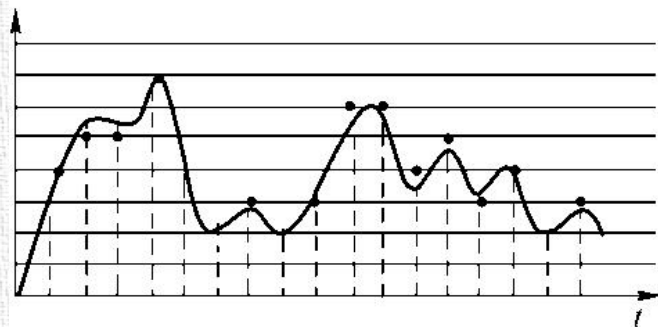
2.5 0与1的声色世界——音频数字化



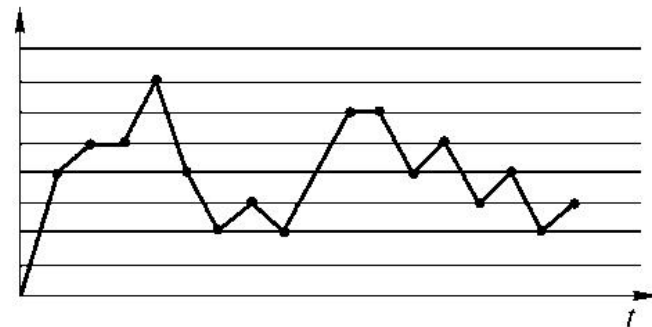
(a) 声音波形，连续的信号



(b) 采样得到的离散值



(c) 量化得到的值



(d) 量化数据还原的波形



2.5 0与1的声色世界——音频数字化



3、数字音频的技术指标

- (1) 采样频率
- (2) 量化位数
- (3) 声道数
- (4) 编码算法



2.5 0与1的声色世界——音频



4、数字音频格式

- WAVE
- MP3文件
- RA文件



2.5 0与1的声色世界——音频



5、电子合成音乐

- 除了用数字音频的方式外，还可以用合成的方式产生音乐。
- 音乐合成的方式根据一定的协议标准，使用音乐符号来记录和解释乐谱，并组成相应的音乐符号，这就是MIDI。
- MIDI是将数字式电子乐器的弹奏过程以命令符号的形式记录下来。当需要播放时，根据记录的乐谱指令，通过音乐合成器生成音乐声波，经过放大后由扬声器播出。



2.5 0与1的声色世界——图形图像



1、图像的种类

- 矢量图
- 位图



(a) 矢量图局部放大



(b) 位图局部放大



2.5 0与1的声色世界——图形图像数字化



- 矢量图也称为面向对象的图像或绘图图像，是计算机图形学中用点、直线或者多边形等基于数学方程的几何图元表示图像。矢量图不存储图像数据的每一点，而是存储图像内容的轮廓部分。例如，一个圆形图案只要存储圆心的坐标位置和半径长度，以及圆形边线和内部的颜色。
- 矢量图形最大的优点是无论放大、缩小或旋转等不会失真，且图像的存储空间较之位图方式要少得多；最大的缺点是难以表现色彩层次丰富的逼真图像效果。矢量图比较适合用于工程图、卡通动漫及图案、标志、VI、文字等设计。



2.5 0与1的声色世界——图形图像数字化



- 位图图像，亦称为点阵图像或栅格图像，图形由排列成若干行、列的像素组成，形成一个像素的阵列，每个像素都具有特定的位置和颜色值。当放大位图时，可以看见赖以构成整个图像的无数单个方块，即像素。这种模式比较适合于内容复杂的图像和真实的照片，但图像在放大和缩小的过程中会失真，占用磁盘空间也较大。
- 总体来看，位图是记录每一个像素的颜色值，再把这些像素点组合成一幅图像；而矢量图是保存节点的位置和曲线、颜色的算法，所以，位图占用的存储空间较矢量图要大的多，而矢量图的显示速度较位图慢。



2.5 0与1的声色世界——图形图像数字化

916

2、图像数字化的过程

— 采样、量化与编码



(a) 原图



(b) 采样



(c) 量化

2.5 0与1的声色世界——图形图像数字化



- 经过采样、量化得到的图像数据量十分巨大，必须采用编码技术来压缩信息量。在一定意义上讲，编码压缩技术是实现图像传输与储存的关键。已有许多成熟的编码算法应用于图像压缩。常见的有图像的预测编码、变换编码、分形编码、小波变换图像压缩编码等。
- 位图是每个格子都独立记录的，因此数据量很大，这就是bmp格式。jpg格式经过一系列运算进行压缩，是有损压缩格式，但画质的损失非常小。



2.5 0与1的声色世界——图形图像



3、图像的基本属性

- 图像分辨率
- 显示分辨率
- 颜色深度



24 位位图



256 色位图



16 色位图



单色位图



2.5 0与1的声色世界——图形图像



4、图像格式

- BMP
- GIF
- TIF
- PCX
- TGA
- JPEG



2.5 0与1的声色世界——视频



1、视频基础知识

视频（Video）就是其内容随时间变化的一组动态图像，所以又叫运动图像或活动图像。它是一种信息量最丰富、直观、生动、具体的承载信息的媒体。

静止的图片称为图像（Image），运动的图像称为视频（Video）。视频信号具有以下特点：

内容随时间的变化而变化；

伴随有与画面同步的声音。



2.5 0与1的声色世界——视频数字化



1、视频基础知识

视频数字化是将模拟视频信号经模数转换和彩色空间变换转为计算机可处理的数字信号，计算机要对输入的模拟视频信息进行采样与量化，并经编码使其变成数字化信息。

在PAL彩色电视制式中采用YUV模型来表示彩色图像。由于所有的显示器都采用RGB值来驱动，所以在显示每个像素之前，需要把YUV彩色分量值转换成RGB值。



2.5 0与1的声色世界——视频数字化



2、视频数字化过程

(1) 采样

由于人的眼睛对颜色的敏感程度远不如对亮度信号灵敏，所以色度信号的采样频率可以比亮度信号的采样频率低，以减少数字视频的数据量。

(2) 量化

采样是把模拟信号变成时间上离散的脉冲信号，而量化则是进行幅度上的离散化处理。

在时间轴的任意一点上量化后的信号电平与原模拟信号电平之间在大多数情况下存在一定的误差，我们通常把量化误差称为量化噪波。

量化位数愈多，层次就分得愈细，量化误差就愈小，视频效果就愈好，但视频的数据量也也就越大。所以在选择量化位数时要综合考虑各方面的因素。



2.5 0与1的声色世界——视频



2、视频数字化过程

(3) 编码

经采样和量化后得到数字视频的数据量将非常大，数据量太大导致保存和传输受限，例如一帧标清分辨率（ 720×576 ）原始视频数据要占用 $720 \times 576 \times 3 = 1244160$ Bytes = 9953280 bits，一秒的实时数据就是近250M bits。所以在编码时要进行压缩。

视频图像数据有极强的相关性，也就是说有大量的冗余信息，具有进行高比例压缩的潜力。其中冗余信息可分为空域冗余信息和时域冗余信息。压缩技术就是将数据中的冗余信息去掉（去除数据之间的相关性），压缩技术包含帧内图像数据压缩技术、帧间图像数据压缩技术和熵编码压缩技术等。



课堂练习



- 计算存储一幅400×300的静态24位真彩图像需要多少字节？如果采用该真彩图像进行视频存储，计算20秒视频（每秒30帧）需要多少字节？

存储时，记录每一个像素点的**RGB**值，对真彩色来讲用**3B**，因此该图像需要的存储空间为：

$$400 \times 300 \times 24 / 8 = 360000 \text{ 字节}$$

计算**20**秒视频（每秒**30**帧）所需的存储空间为：

$$360000 \times 30 \times 20 = 216000000 \text{ 字节}$$



2.5 0与1的声色世界——视频



3、 常见数字视频格式

- (1) AVI
- (2) DV
- (3) MOV
- (4) MPEG
- (5) REAL VIDEO





Thanks !

